Load Balancing & Resource Utilization in Cloud

Distributing work to keep cloud systems fast, efficient and available

Department of Computer Science and Engineering

Why Load Balancing Matters

1 **Performance**

Spreads requests so no server becomes a bottleneck

2 **Availability**

Reroutes traffic away from failed or unhealthy nodes

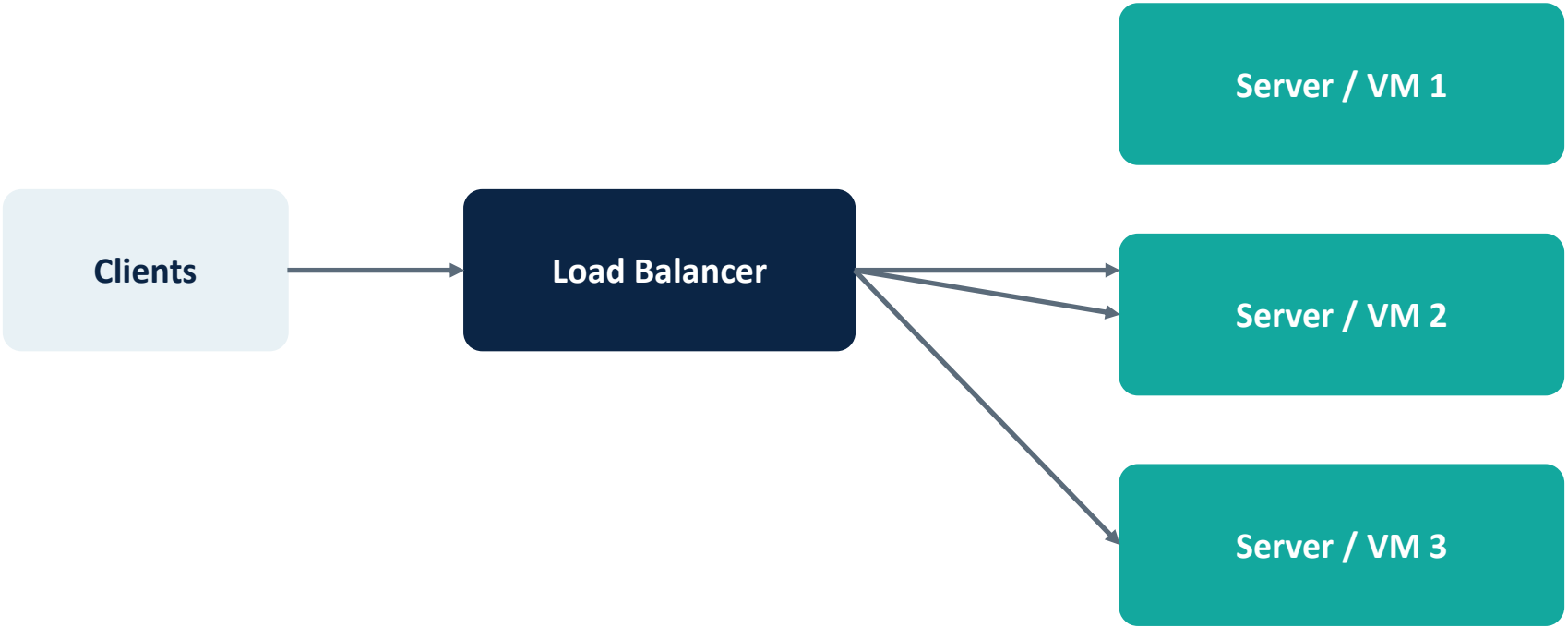
3 **Scalability**

Adds or removes capacity as demand changes

4 **Cost efficiency**

Higher utilization means fewer idle resources to pay for

How It Works



Health checks, session persistence and routing policy decide where each request goes.

Layer 4 vs Layer 7 Load Balancing

Layer 4 (Transport)

Routes on IP address and port

- Very fast, low overhead
- Protocol-agnostic (TCP/UDP)
- No view into request content
- Good for raw throughput

Layer 7 (Application)

Routes on HTTP content

- Path, host and header-based routing
- SSL termination and caching
- Cookie-based session affinity
- Higher CPU cost, richer control

Load Balancing Algorithms

Type	Algorithm	Best suited for
Static	Round Robin	Uniform servers, similar requests
Static	Weighted Round Robin	Servers with unequal capacity
Dynamic	Least Connections	Long-lived, uneven sessions
Dynamic	Least Response Time	Latency-sensitive services
Hash-based	IP Hash	Session affinity without state sharing
Adaptive	Predictive / ML-based	Bursty, forecastable workloads

Algorithm Mechanics

Weighted Round Robin

`share_i = w_i / Σw`

Weights reflect capacity, e.g. 4-core = 2, 2-core = 1.

Least Connections

`argmin(active_i / w_i)`

Tracks open connections; weight-normalized for mixed hardware.

Power of Two Choices

`sample 2, pick lighter`

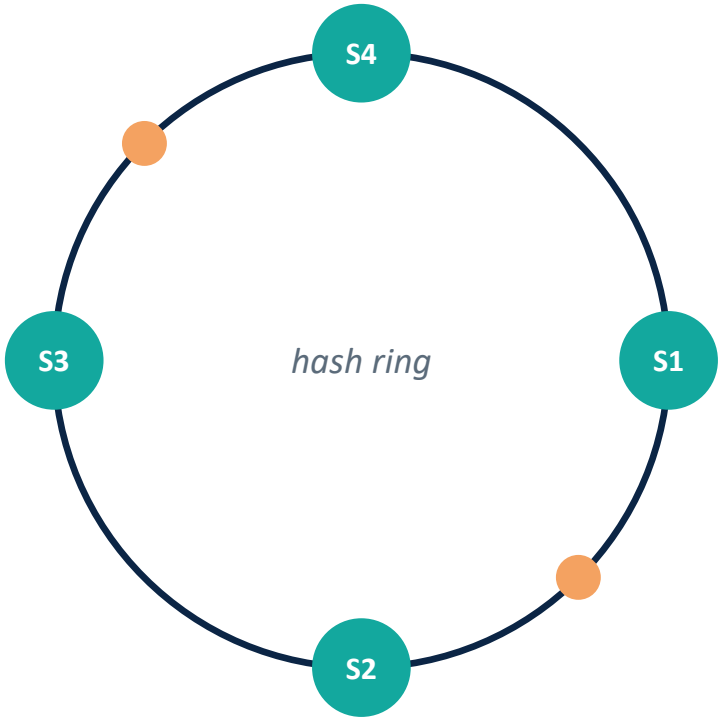
Avoids scanning every server; max load falls sharply vs pure random.

Peak EWMA Latency

`ewma_rtt × (inflight+1)`

Favors fast, lightly loaded backends and adapts to slow nodes.

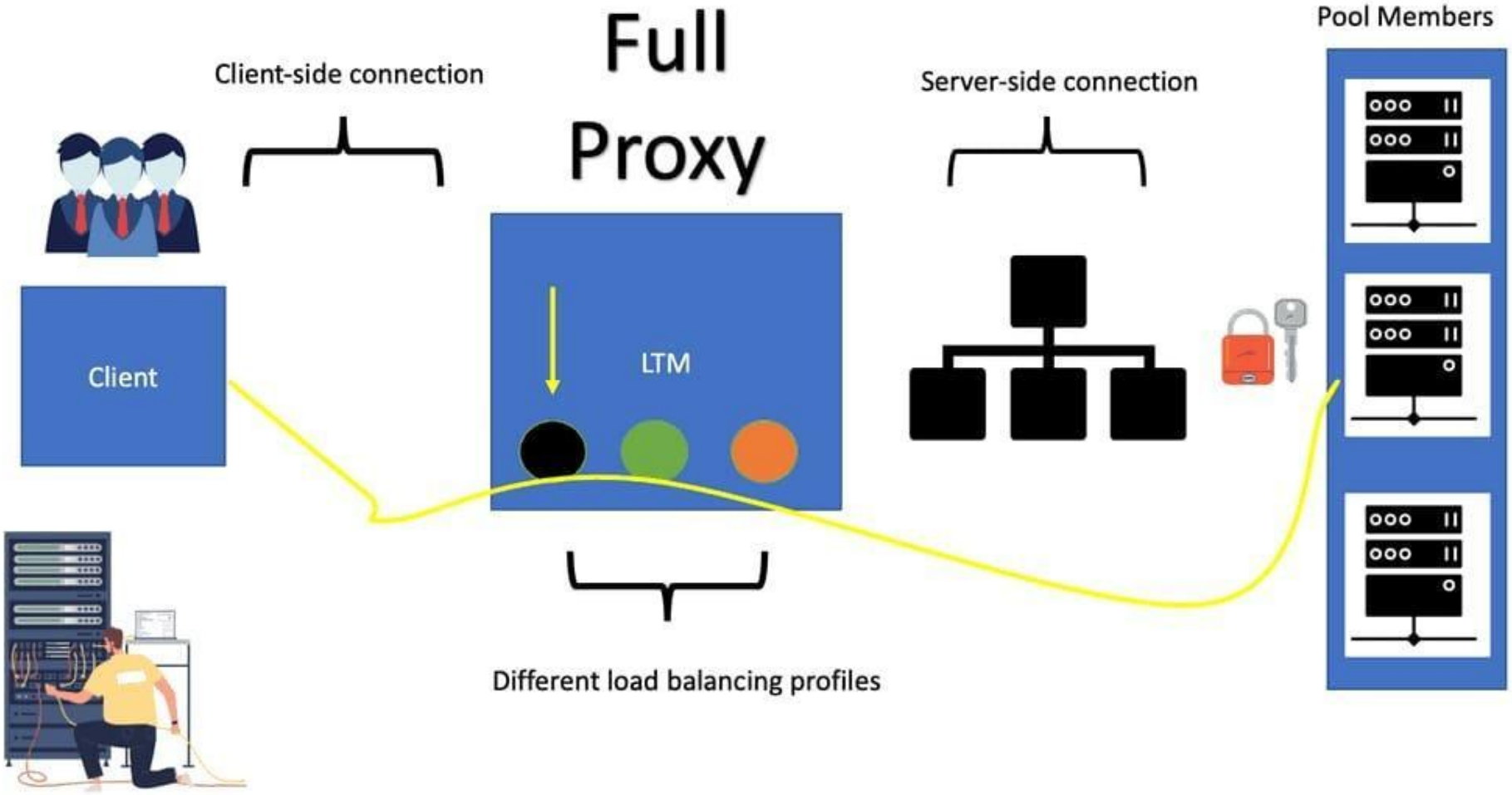
Consistent Hashing



- hash(key) mod N: adding one server remaps most keys
- Ring: each key goes to the next server clockwise (amber dots = keys)
- Adding or removing a node moves only about 1/N of keys
- Virtual nodes even out uneven arcs
- Maglev-style lookup tables trade a little balance for fast, stable mapping

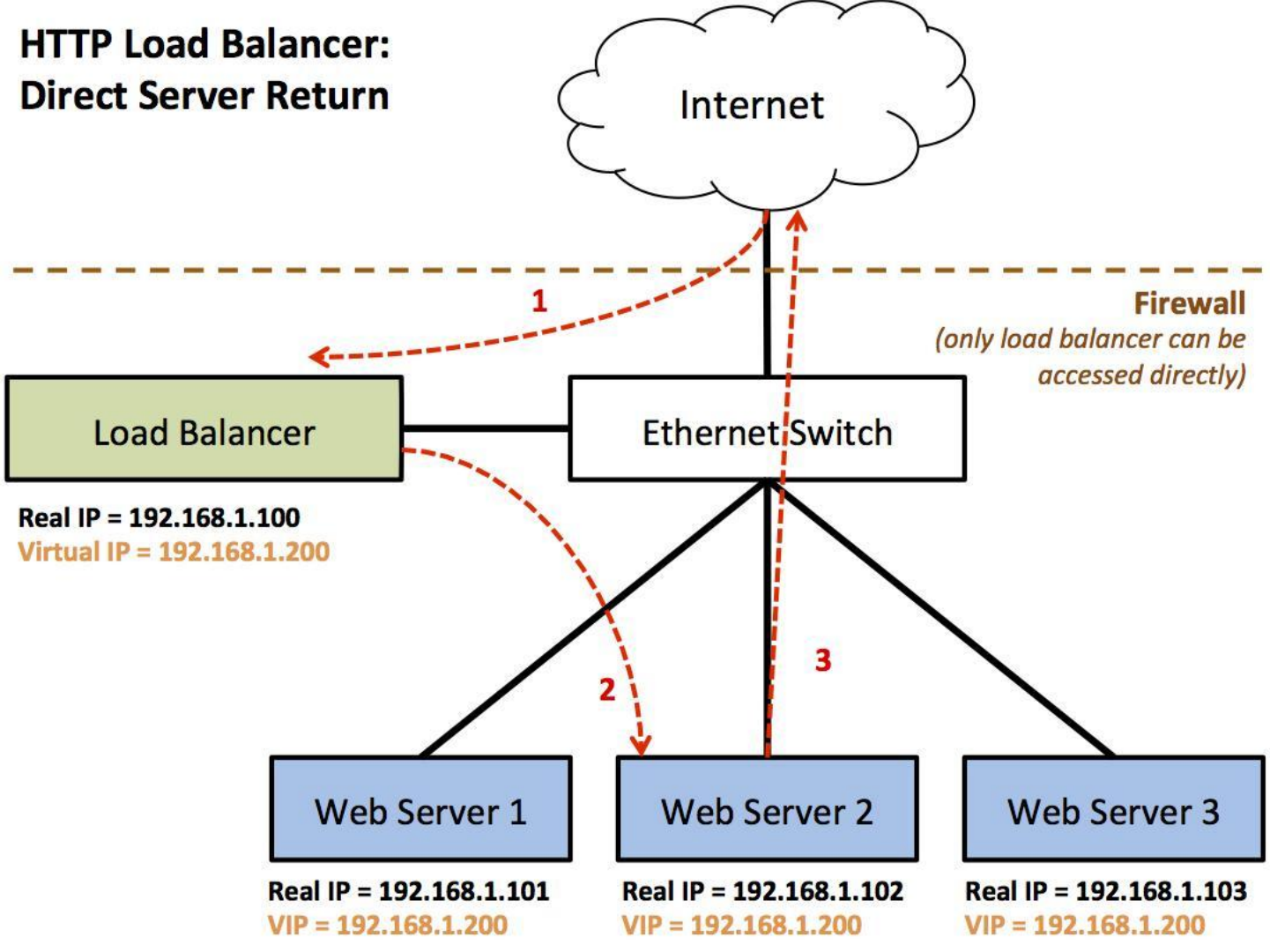
Forwarding Modes and Balancer Topologies

Mode	How it works	Trade-off
Proxy (full NAT)	LB terminates the client connection and opens a new one to the backend	Full L7 control; all traffic in both directions crosses the LB
Direct Server Return	LB rewrites the destination MAC; backend replies straight to the client	High throughput; no L7 features, VIP on loopback
IP tunneling	LB encapsulates packets (IP-in-IP/GRE) to backends	Works across subnets; adds MTU overhead
ECMP + anycast	Routers spread flows over LB instances by 5-tuple hash	Scales LBs horizontally; flows can remap when instances change
DNS / GSLB	Name resolves to different regional addresses	Global steering; failover limited by TTL


Full
Proxy
Client-side connection
Server-side connection
Pool Members
Client
LTM
Different load balancing profiles

NGINX AS REVERSE PROXY & LOAD BALANCER




HTTP Load Balancer:
Direct Server Return
Internet
1
Firewall
(only load balancer can be accessed directly)
Load Balancer
Ethernet Switch
Real IP = 192.168.1.100
Virtual IP = 192.168.1.200
2
3
Web Server 1
Web Server 2
Web Server 3
Real IP = 192.168.1.101
VIP = 192.168.1.200
Real IP = 192.168.1.102
VIP = 192.168.1.200
Real IP = 192.168.1.103
VIP = 192.168.1.200

Health Checks and Failure Detection

Parameter	Example
Probe interval	10 s
Probe timeout	5 s
Unhealthy threshold	3 failures
Healthy threshold	2 successes

- Active probes: TCP connect or HTTP GET /healthz expecting 200
- Passive checks: eject nodes after consecutive 5xx or timeouts (outlier detection)
- Connection draining: let in-flight requests finish before removal
- Keep health endpoints cheap; deep dependency checks can cause cascading ejections

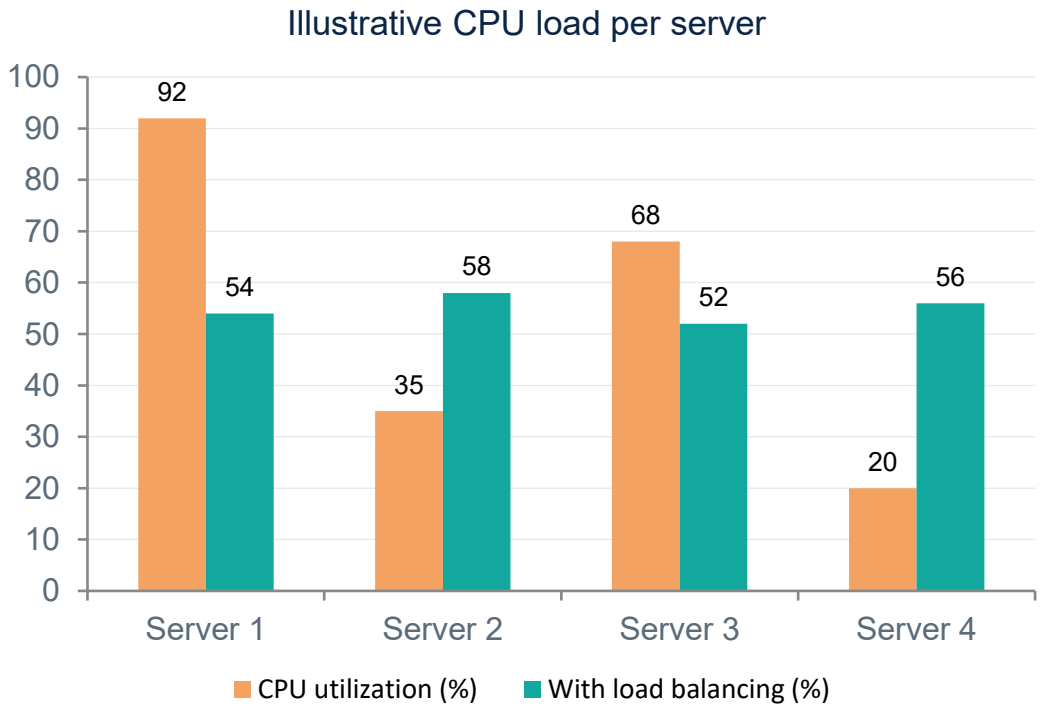
Worst-case detection ≈ interval × unhealthy threshold ≈ 30 s

Session Persistence and State

Method	Mechanism	Drawback
Source-IP affinity	Hash of client IP picks the backend	Clients behind one NAT pile onto one server
LB-inserted cookie	LB sets a cookie naming the backend (L7)	Session lost if that backend is removed
App session ID	LB routes on an application cookie	LB must understand the app protocol
Stateless + shared store	Sessions kept in Redis or a database; any node serves	Extra latency; the store becomes a dependency

Prefer stateless services: balancing, scaling and failover all become simpler.

Impact on Resource Utilization



Without balancing
Hot spots and idle nodes coexist, wasting capacity.

With balancing
Load evens out, so the same fleet handles more traffic.

Illustrative data

Quantifying Load Imbalance

Imbalance factor

`max(load) / mean(load)`

1.0 means perfectly even.

Jain's fairness index

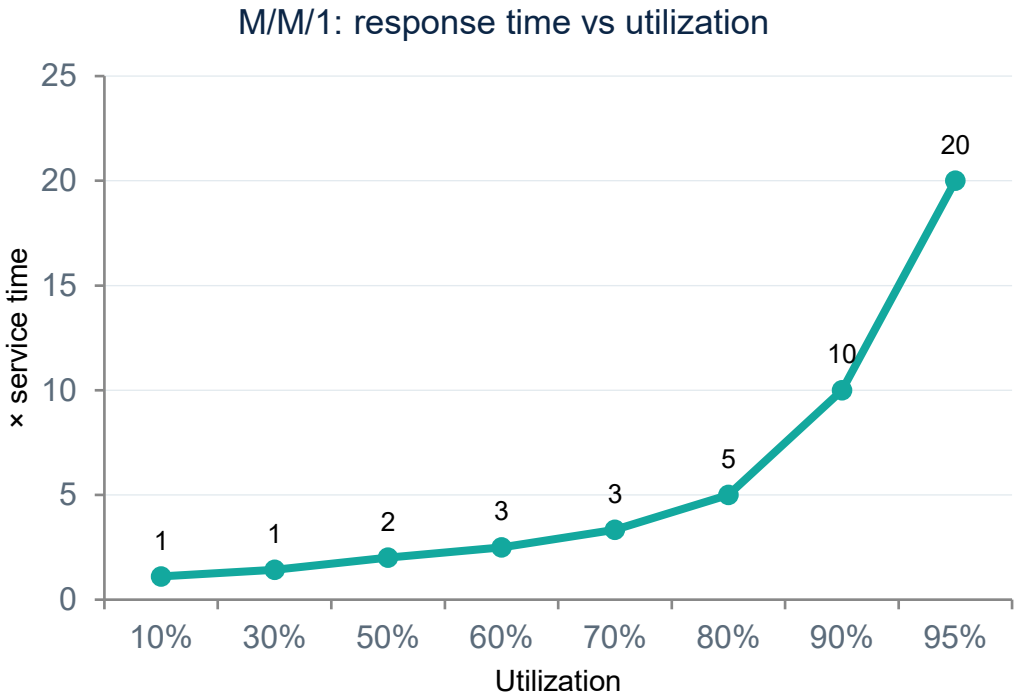
$J = (\Sigma x)^2 / (n \cdot \Sigma x^2)$

Ranges from 1/n (worst) to 1 (even).

Metric	Before	After
Max / mean	1.71	1.05
Jain's index	0.79	1.00
Std dev (pp)	28.1	2.2

Computed from the illustrative CPU values on the utilization chart.

Why Headroom Matters: A Queueing View



$W = (1/\mu) / (1 - \rho)$

Little's Law: $L = \lambda \cdot W$

Delay grows without bound as ρ approaches 1, so plan capacity for peaks, not averages.

Challenges and Optimization Techniques

Challenges

- Unpredictable, bursty traffic
- Heterogeneous VMs and hardware
- Balancer as a single point of failure
- Migration overhead and SLA limits

Techniques

- Auto-scaling with health checks
- VM migration and consolidation
- Multi-tier and global balancing
- Energy-aware and ML-driven scheduling

Load Balancing in Major Clouds

1	**AWS**	**Elastic Load Balancing** Application, Network and Gateway load balancers
2	**Microsoft Azure**	**Load Balancer, Application Gateway** Layer 4 balancing plus Layer 7 gateway; Front Door for global routing
3	**Google Cloud**	**Cloud Load Balancing** Global and regional balancing behind a single anycast IP

Service names as commonly offered; check provider documentation for current options.

Measuring Resource Utilization

CPU

Processing load per VM or container

Memory

Working set vs allocated RAM

Network I/O

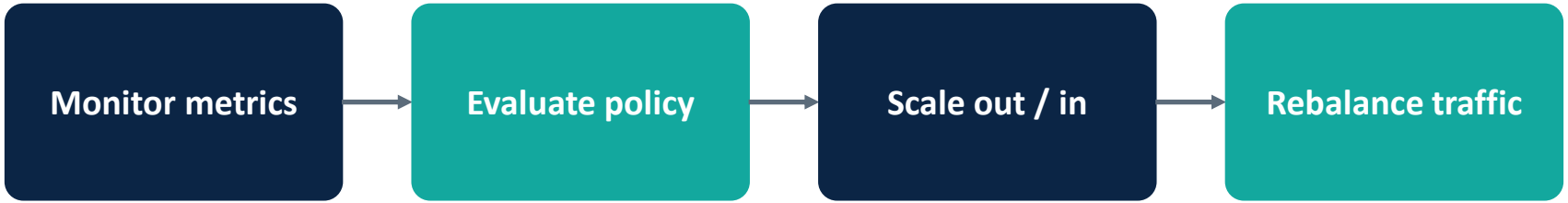
Bandwidth and packets per second

Response time

Latency and request queue depth

Goal: keep average utilization high enough to avoid waste, with headroom to absorb spikes and meet SLAs.

Auto-Scaling: The Feedback Loop



Scale out when load crosses an upper threshold; **scale in** when it stays low, using cooldown periods to avoid flapping.

Auto-Scaling: Target Tracking Math

$$\text{desired} = \text{ceil}(\text{current} \times \text{metric} / \text{target})$$

4 instances at 80% CPU, target 50% → ceil(4 × 1.6) = 7

Cooldown

Wait after a scaling action before acting again.

Stabilization

Use recent recommendations to damp flapping, especially on scale-in.

Warm-up

Exclude new instances from metrics until they are ready.

VM Placement and Live Migration

Placement as bin packing

- NP-hard: pack VMs onto the fewest hosts under CPU, RAM and network limits
- Heuristics: First Fit, Best Fit, First Fit Decreasing
- Metaheuristics: genetic algorithm, PSO, ant colony
- Trade-off: consolidation saves energy but raises contention risk

Live migration (pre-copy)

- Copy memory pages while the VM keeps running
- Re-copy pages dirtied meanwhile, in rounds
- Pause briefly; send last dirty pages and CPU state
- Resume on the destination host
- Cost: network and CPU; high dirty-page rates prolong migration

Best Practices

1. Run balancers redundantly across zones
2. Configure health checks and graceful draining
3. Design services to be stateless where possible
4. Terminate SSL at the balancer and monitor continuously
5. Load test and tune scaling thresholds regularly

Key Takeaways

- Load balancing keeps cloud services responsive and available
- The right algorithm depends on workload and server heterogeneity
- Better utilization cuts cost and energy use
- Future direction: adaptive, ML-based and energy-aware balancing

Thank you | Questions?